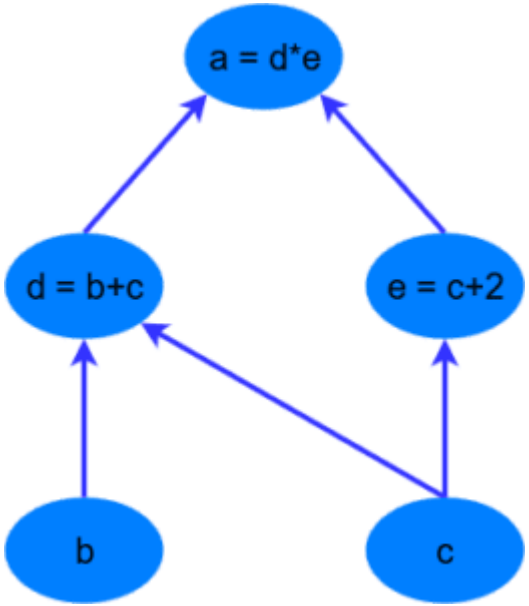
Особенности практического использования нейронных сетей

Задание 1. Оптимизация на Pytorch.

Дмитрий Буряк
к.ф.-м.н
dyb04@yandex.ru



Динамический граф вычислений



*a = (b + c) * (c + 2)*

- ❑ Выражение разбивается на части, которые вычисляются последовательно.
- ❑ Каждый узел графа: независимый кусок кода, которому для работы нужны лишь входы.
- ❑ Преимущества: параллельная обработка.
- ❑ Технически: тяжелые вычисления (свертки, умножения матриц) реализованы на C/C++ / CUDA C++.

Тензоры

❑ Основная вычислительная единица - тензор, можно представить как вложение произвольного количества динамических массивов переменной размерности.

❑ Инициализация: 3 матрицы размера 2x2

```
In [1]: import torch
In [2]: x=torch.Tensor(3,2,2)
In [3]: print(x)
tensor([[[8.7725e+29, 3.0904e-41],
         [0.0000e+00, 0.0000e+00]],

        [[       nan, 0.0000e+00],
         [1.3733e-14, 6.4069e+02]],

        [[4.3066e+21, 1.1824e+22],
         [4.3066e+21, 6.3828e+28]]])
```

❑ Матричное умножение

```
In [18]: a=torch.rand(2,3)
In [19]: b=torch.rand(3,2)
In [20]: c=torch.mm(b,a)
In [21]: print(c)
Out[21]:
tensor([[0.4033, 0.0286, 0.1627],
        [0.6677, 0.0475, 0.2699],
        [0.5106, 0.0588, 0.3515]])
```

```
In [38]: a=a*0+1
In [39]: b=b*0+2
In [40]: 3+torch.mm(b,a)
Out[40]:
tensor([[7., 7., 7.],
        [7., 7., 7.],
        [7., 7., 7.]])
```

Тензоры и вычислительные устройства

- 2 типа: на CPU и на GPU

```
In [73]: a=torch.rand(3,3)
In [74]: a
Out[74]:
tensor([[0.5106, 0.9224, 0.0468],
        [0.0877, 0.6954, 0.2929],
        [0.0226, 0.1924, 0.0517]])
In [75]: a=a.cuda(device = 0)
In [76]: a
Out[76]:
tensor([[0.5106, 0.9224, 0.0468],
        [0.0877, 0.6954, 0.2929],
        [0.0226, 0.1924, 0.0517]],
device='cuda:0')
In [77]: a=a.cpu()
In [78]: a
Out[78]:
tensor([[0.5106, 0.9224, 0.0468],
        [0.0877, 0.6954, 0.2929],
        [0.0226, 0.1924, 0.0517]])
```

```
In [79]: a=torch.rand(3,3)

In [80]: b=torch.rand(3,3).cuda(0)

In [81]: a+b
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
<ipython-input-81-ca730b97bf8a> in <module>
----> 1 a+b
RuntimeError: expected type torch.FloatTensor but got
torch.cuda.FloatTensor
```

- Операции между тензорами должны выполняться на одном устройстве

  torch.cuda.is_available()

  torch.cuda.device_count()

Автоматическое дифференцирование

```
In [42]: from torch.autograd import Variable
In [43]: x=Variable(torch.ones(2,2)*2,requires_grad = True)
In [44]: z = 2 * (x * x) + 5 * x
In [45]: z.backward(torch.ones(2, 2))
In [46]: print(x.grad)
tensor([[13., 13.],
        [13., 13.]])
In [47]: z
Out[47]:
tensor([[18., 18.],
        [18., 18.]], grad_fn=<AddBackward0>)
```

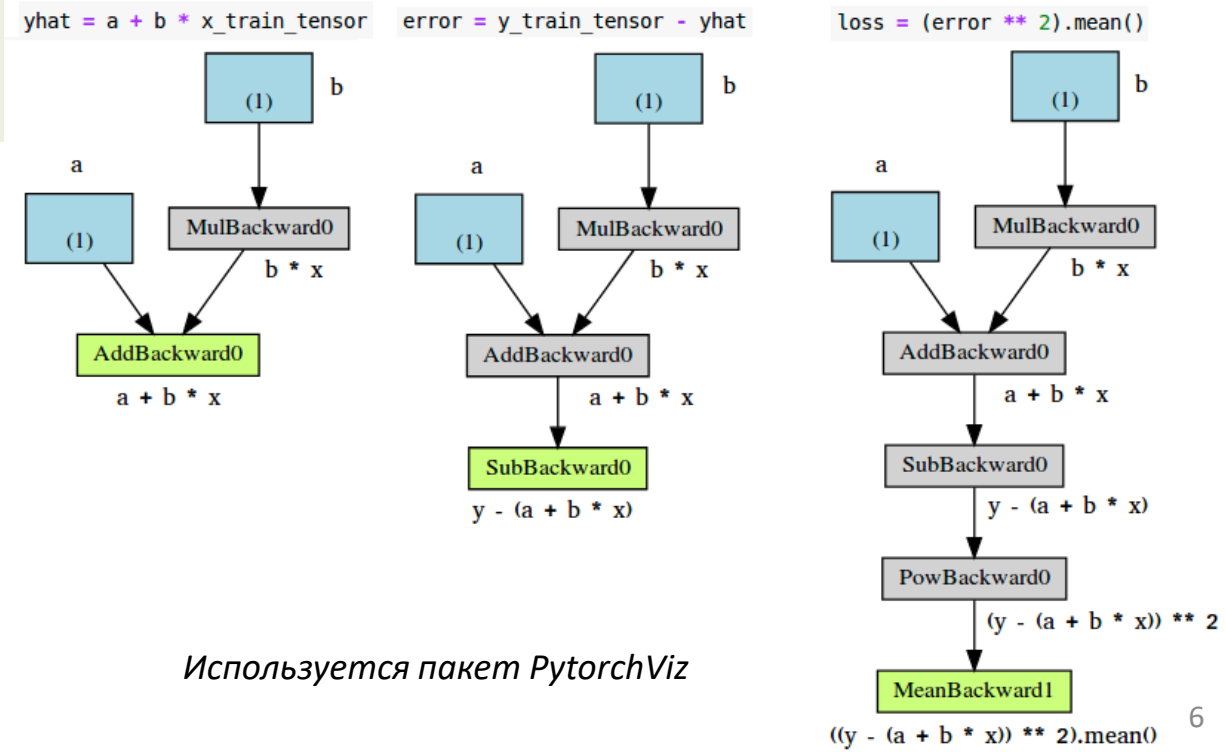
dz/dx = 4x+5,
При x_{ij} =2 dz/dx должен быть заполнен числами 13

Компонент тензора .grad содержит градиент:
значение или функцию

- Как это реализовано: все математические операции (+,-,*,/…) с тензорами перегружены.
- Каждая операция с тензорами проверяет, нужно ли вычислять градиент (компонент класса torch.Tensor requires_grad == True)
- Каждый torch.Tensor содержит компоненты:
  grad (значение градиента) или
  grad_fn(ссылка на функцию вычисления градиента)
- Вызов z.backward() запускает проход по графу вычислений в обратную сторону, с вызовом функций grad_fn

Примеры вычислительных графов в представлении Pytorch (1)

```
a = torch.randn(1, requires_grad=True, dtype=torch.float, device=device)
b = torch.randn(1, requires_grad=True, dtype=torch.float, device=device)
yhat = a + b * x_train_tensor
error = y_train_tensor - yhat
loss = (error ** 2).mean()
```

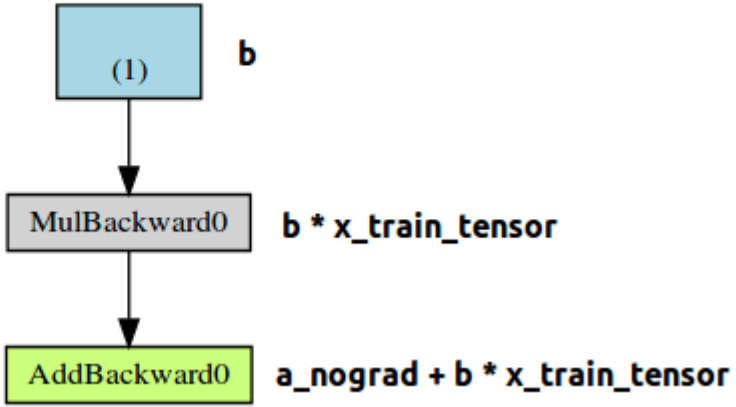


Используется пакет PytorchViz

Примеры вычислительных графов в представлении Pytorch (2)

```
a_nograd = torch.randn(1, requires_grad=False, dtype=torch.float, device=device)
b = torch.randn(1, requires_grad=True, dtype=torch.float, device=device)

yhat = a_nograd + b * x_train_tensor
```



Пример. Линейная регрессия (1)

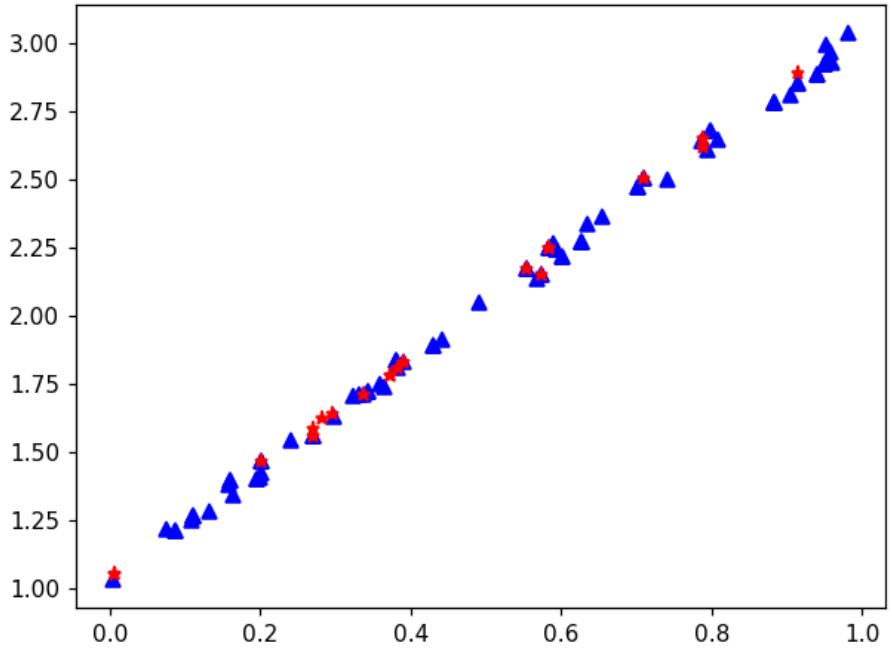
```
import numpy
import torch
import torch.optim as optim
from matplotlib import pyplot as plt

# Data Generation
torch.manual_seed(42)
x = torch.rand(100, 1)
y = 1 + 2 * x + 0.1 * torch.rand(100, 1)

# Shuffles the indices
idx = (torch.rand(100)*100).long()

# Train-valid split
train_idx = idx[:80]
val_idx = idx[80:]
x_train, y_train = x[train_idx], y[train_idx]
x_val, y_val = x[val_idx], y[val_idx]

plt.figure("train")
plt.plot(x_train.numpy(),y_train.numpy(),"b",lw=0,marker="^")
plt.plot(x_val.numpy(),y_val.numpy(),"r",lw=0,marker="*")
plt.show()
```



Пример. Линейная регрессия (2)

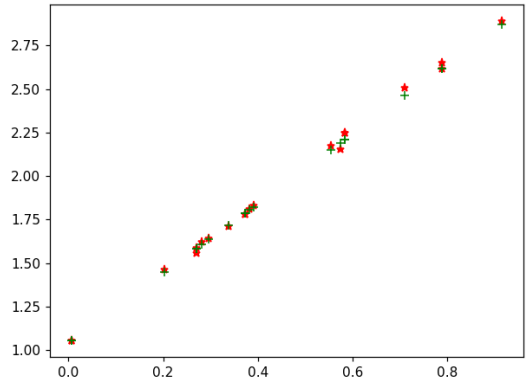
```
torch.manual_seed(42)
a = torch.randn(1, requires_grad=True, dtype=torch.float)
b = torch.randn(1, requires_grad=True, dtype=torch.float)
print("Initial values:",a, b)
lr = 1e-1
n_epochs = 1000
# Defines a SGD optimizer to update the parameters
optimizer = optim.SGD([a, b], lr=lr)

for epoch in range(n_epochs):
    yhat = a + b * x_train
    error = y_train - yhat
    loss = (error ** 2).mean()

    loss.backward()

    #     a -= lr * a.grad
    #     b -= lr * b.grad
    optimizer.step()
    optimizer.zero_grad()
```

```
print("optimized values:",a,b)
with torch.no_grad():
    yhat = a + b * x_train
    yres = a + b * x_val
print("train error:",torch.mean((y_train - yhat)**2))
print("validation error:",torch.mean((y_val- yres)**2))
plt.figure("result")
plt.plot(x_val.numpy(),y_val.numpy(),"r",lw=0,marker="*")
plt.plot(x_val.numpy(),yres.numpy(),"g",lw=0,marker="+")
plt.show()
```




```
Initial values: tensor([0.3367], requires_grad=True) tensor([0.1288], requires_grad=True)
optimized values: tensor([1.0422], requires_grad=True) tensor([1.9991], requires_grad=True)
train error: tensor(0.0007)
validation error: tensor(0.0005)
```

Задание

Реализовать регрессию квадратичной функции

- Провести эксперименты
  . с разным уровнем шума для данных
  . варианты инициализации оптимизируемых параметров: 0, из N(0,1), из U(1000,2000)
  . Скорость обучения: 0.001, 0.01, 0.1, 0.5, 0.7, 0.9
- Подготовить отчет: описание экспериментов, графики, результаты (ошибка, найденные параметры), исходный код.
- Заявленные результаты должны быть воспроизводимы с помощью предоставленного исходного кода.
- **Срок выполнения: 16.11.2026**